Technische Universität München
Wind Energy Institute

Praktikum
Design of Wind Farms

Lecture 4:
Wake Models, FLORIS

Hadi Hoghooghi
Technische Universität München

2026

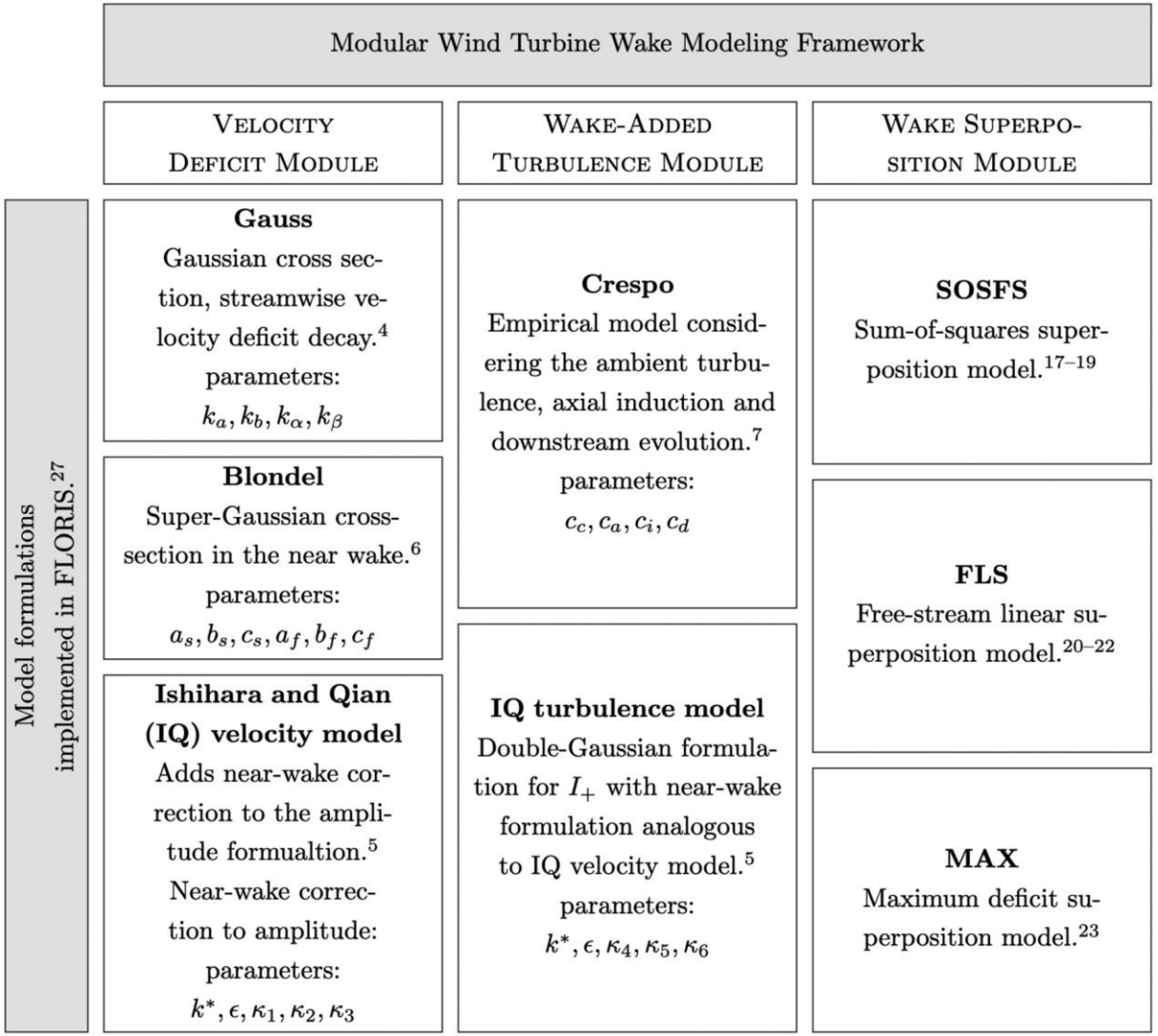
Syllabus

No.	Week	Date (Wed)	9.30–10.15	10.30–11.15	11.30–12.15
1	14	15/04/26	Overview of Course **Lecture 1**: Background and Motivations	**Lecture 2**: Development of a Wind Farm Project	**Lecture 3**: Wake Behavior, Wind Farm Control **Assignment 1**: Basics and Recapitulation
2	15	22/04/26	**Lecture 4**: Wake Models, FLORIS	**Assignment 1**: Solution **Assignment 2:** Jensen's Wake Model and Wake Superposition in Wind Farms	**Tutorial 1:** FLORIS Installation and Setup
3	16	29/04/26	**Lecture 5_1**: LandBOSSE	**Tutorial 2:** LANDBOSSE Installation and Setup	**Assignment 2**: Solution **Assignment 3**: Wind Farm Analysis Using FLORIS
5	17	06/05/26	**Lecture 5_2**: Financial Analysis	**Assignment 3:** Solution	**Assignment 4**: Wind Farm Cost Analysis Using FLORIS + LandBOSSE)
6	18	13/05/26	**Lecture 6**: Layout Optimization	**Assignment 4**: Solution	**Assignment 5:** Layout Optimization
7	19	20/05/26	**Assignment 5**: Solution	**Final group assignment**: Introduction, Finalizing Groups, Q&A	
8	20	27/05/26	**Final group assignment:** Design Process, Q&A, Writing Report		
9	21	03/06/26	**No Class**		
10	22	10/06/26	**Final group assignment:** Design Process, Q&A, Writing Report		
11	28	08/07/26	**No Class** **Deadline:** Electronic report submission until **Tuesday, 07/07/2026, 23:59h**		
12	29	15/07/26	**Final presentation – 8:30h (Room 2701M)**		

Agenda

- Wake Models
- Wake Superposition Models
- Overview on FLORIS Wake Models
- FLORIS Examples

Wake Model Components



Ref: [1]

Velocity Deficit Models

- Based on **Gaussian wake** model, the velocity deficit in each model is represented:

$$\frac{\Delta U}{U_\infty} = C(x)\ \exp\left(-\frac{r^n}{2\sigma^2}\right) \qquad r = \sqrt{y^2 + (z-H)^2}$$

velocity deficit

streamwise

standard deviation of the distribution

spanwise

vertical

Hub height

$$\left(\frac{\Delta U}{U_\infty}\right)_{\max} = C(x) = 1 - \sqrt{1 - \frac{C_T}{8(k^* x/D + \epsilon)^2}}$$

Thrust coeff.

Decays moving downstream

$$k^* = \partial\sigma/\partial x$$

$$\epsilon = \lim_{x\to 0} \sigma/D = 0.25\beta \qquad \beta = \frac{1+\sqrt{1-C_T}}{2\sqrt{1-C_T}}$$

Velocity Deficit Models

- **Niayifar and Porte-Agel:** $k^* = 0.3837I + 0.003678$

  local turbulence intensity

- **FLORIS:** $k^* = k_y = k_z = k_a \cdot I + k_b$

The Gaussian model in FLORIS is only considered to be applicable in the **far wake**, which satisfies the condition, where k_α and k_β are parameters tuned empirically.

$$x_{fw} \geq D\frac{1+\sqrt{1-C_T}}{\sqrt{2}(4k_\alpha I + 2k_\beta * \left(1-\sqrt{1-C_T}\right))}$$

- **Blondel and Cathelain:** a near wake correction $k^* = c_{\mathrm{NW}}(1+x)^{p_{\mathrm{NW}}}$

$$p_{\mathrm{NW}} = -1 \qquad a = (1-\sqrt{1-C_T})/2$$

$$c_{\mathrm{NW}} = \sqrt{\frac{C_T}{8(1-(1-a)^2)}} - c_s\sqrt{\beta}$$

a_s, b_s, and c_s are tuning coefficients fit to large eddy simulation data

$$\epsilon = (a_s I + b_s)x/D + c_s\sqrt{\beta}$$

- **Ishihara and Qian:** an alternative correction

Wake-added Turbulence Models

- Purely **empirical wake-added turbulence model** that relies on:
  - The thrust coefficient alone in the near wake (defined as x/D < 3 for that case)
  - The thrust coefficient and the ambient turbulence in the far wake

Wake-added Turbulence ⟷ $$I_+ = \begin{cases} c_c a & \text{if } x/D < 3 \\ c_c a^{c_a} I^{c_i} (x/D)^{c_d} & \text{if } x/D \geq 3 \end{cases}$$

- where the constant coefficients c_c, c_a, c_i, and c_d modulate the dependence on the scale (constant), axial induction factor, ambient turbulence intensity, and downstream distance, respectively.

$$I_+ = \frac{1}{(\kappa_4 + \kappa_5 x/D + \kappa_6 (1 + x/D)^{-2})^2} \times \left[k_1 \exp\left(-\frac{(r - D/2)^2}{2\sigma} \right) + k_2 \exp\left(-\frac{(r + D/2)^2}{2\sigma} \right) \right] - \delta(z)$$

$$k_1 = \begin{cases} \cos^2(\pi(r/D - 0.5)/2) & \text{if } r/D \leq 0.5 \\ 1 & \text{if } r/D > 0.5, \end{cases}$$

$$k_2 = \begin{cases} \cos^2(\pi(r/D + 0.5)/2) & \text{if } r/D \leq 0.5 \\ 0 & \text{if } r/D > 0.5. \end{cases}$$

tuning coefficients, k_4, k_5, and k_6

$$\delta(z) = \begin{cases} 0 & \text{if } z/H \geq 1 \\ I \sin^2(\pi((H - z)/H) & \text{if } z/H < 1 \end{cases}$$

Wake Superposition Models

- **Lissaman**: combining velocity deficits **linearly**

$$U_i(x) = U_\infty - \sum_{i=1}^{N} \Delta U_i \cdot U_\infty$$

$$\Delta U_i(x) = (1 - U_i(x)/U_{\infty,i})$$

- **Superposition of energy deficits**, rather than velocity or momentum, leading to a root-sum-squares formulation:

$$U_i(x) = U_\infty - \sqrt{\sum_{i=1}^{N} \Delta U_i^2 \cdot U_\infty^2}$$

- Maximum velocity deficit:

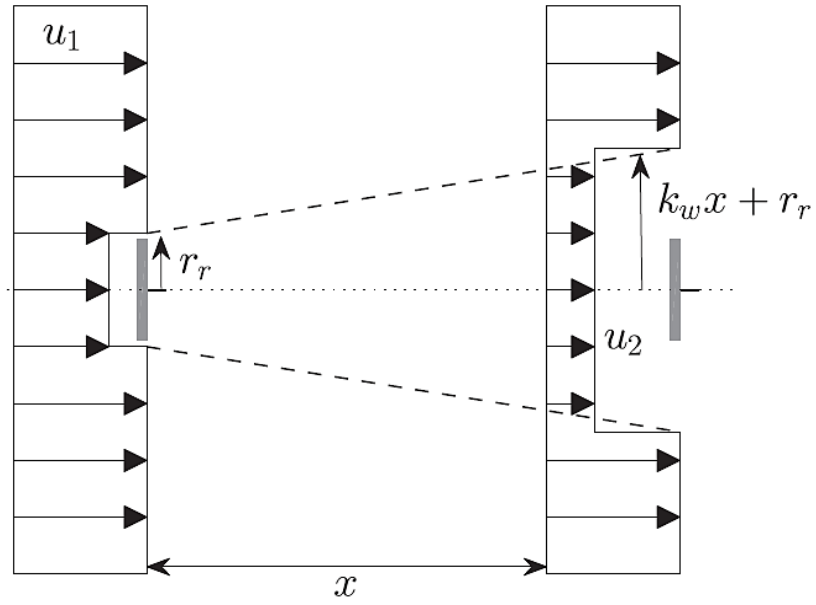
$$U_i(x) = U_\infty - \max_i(\Delta U_i \cdot U_\infty)$$

Jensen Wake Model

- **Formulated in the early 1980s, simple, very fast and easy to implement**

$$1 - \frac{u_2}{u_1} = \frac{1 - \sqrt{1 - C_t}}{(1 + k_w\, x/r_r)^2}$$

- where C_t is the thrust coefficient, r_r is the rotor radius and k_w is the wake decay coefficient ($k_w = 0.1$)
- Suggested values for **k_w = 0.07 5** for on-shore and **0.04/0.05** for offshore
- within a wind farm, the **speed deficit** at the **nth** turbine



$$\delta_n = \left(\sum_{i=1}^{n} \delta_i^2 \right)^{1/2}$$

- The speed at the **n_{th}** turbine, **u_n**, is then given as:

$$u_n = u_1 \,(1 - \delta_n)$$

Bastankhah Analytical Wake Model

- **A new analytical wake model;** neglecting viscous and pressure terms in the momentum equation

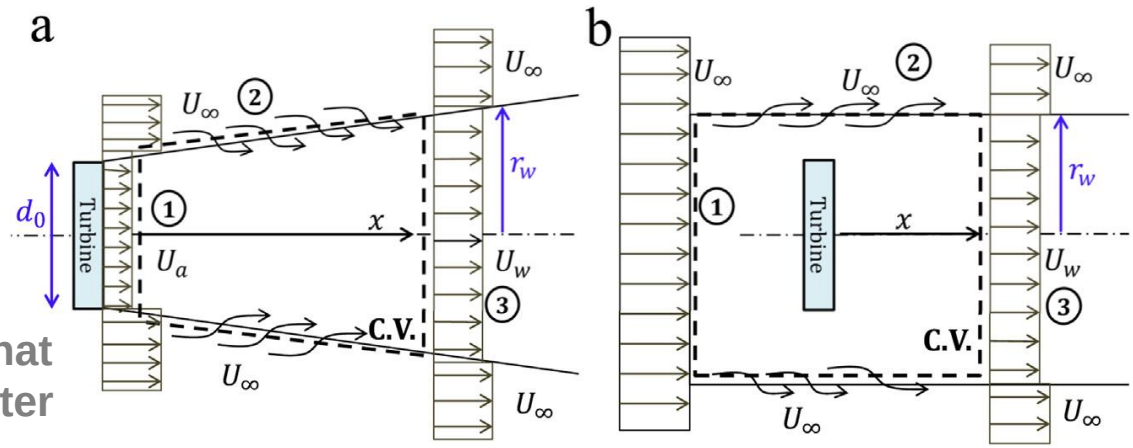
- **y** and **z** are spanwise and vertical coordinates, respectively,
- **z_h** is the hub height,

$$\frac{\Delta U}{U_\infty} = \left(1 - \sqrt{1 - \frac{C_T}{8(k^* x/d_0 + \varepsilon)^2}}\right) \times \exp\left(-\frac{1}{2(k^* x/d_0 + \varepsilon)^2}\left\{\left(\frac{z - z_h}{d_0}\right)^2 + \left(\frac{y}{d_0}\right)^2\right\}\right)$$

$$\beta = \frac{1}{2}\frac{1 + \sqrt{1 - C_T}}{\sqrt{1 - C_T}}$$

$$\varepsilon = 0.2\sqrt{\beta}$$

- **k*** is the wake growth rate that need to specify one parameter (value of k*) for each case.

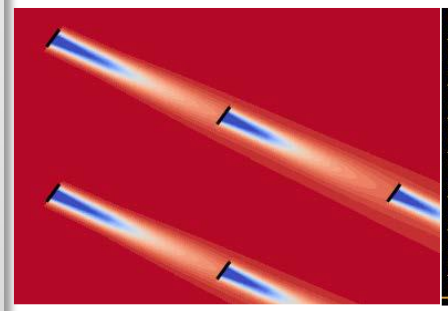


FLOw Redirection and Induction in Steady State

FLORIS

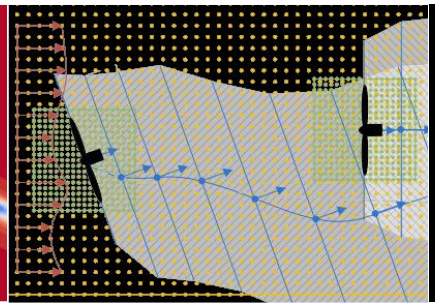
Modeling Tools at NREL

Increasing flow physics



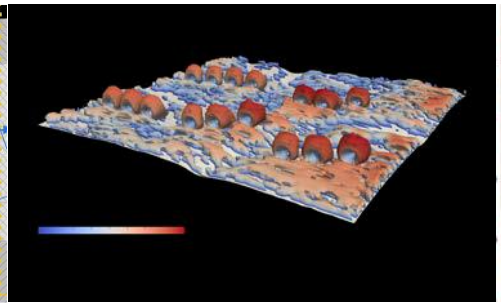
FLORIS

- Control-oriented model
- **Runs in fractions of seconds**
- Can be used to find optimal control settings and analyse across wind rose



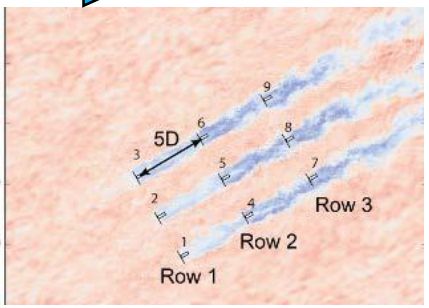
FAST.FARM

- New code which overlays DWM) wakes
- Includes embedded FAST models of turbines
- **Runs on few cores**, near real time, allowing load suite analysis



WindSE

- Solves the steady/ unsteady 2D/3D RANS equations
- Adjoints included for large-scale optimizations
- **Runs in serial or in parallel**, in minutes



SOWFA

- Wind farm simulator based on large-eddy simulation
- Allows detailed investigation of wake physics, but **requires many cores and time to run simulations**

All are (will be) available open source on **www.github.com**

Ref: NREL

FLORIS: Controls-oriented Wind Farm Model

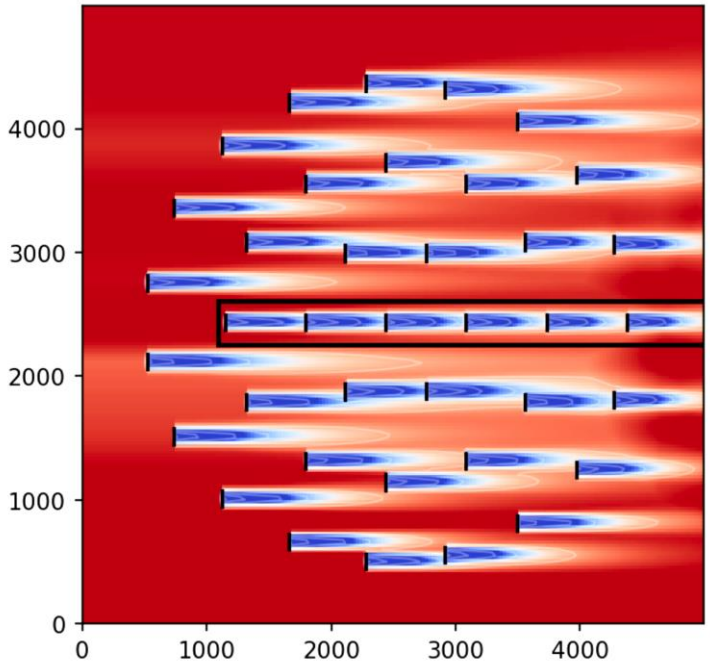
- FLORIS (FLOw Redirection and Induction in Steady State) is an open-source framework, developed originally by the **National Renewable Energy Laboratory (NREL)** and **TU Delft**, which includes both **control-oriented wake models** as well as tools used for the **design and analysis of wind farm control strategies**.
- Computationally inexpensive (<1s for 100 turbines)

Models

- **Wake models**
- **Turbulence models**
- **Turbine models**

Tools

- **Visualization**
- **Optimization**
- **Analysis**



- FLORIS is open-source and shared freely on github.com at: https://github.com/NREL/floris

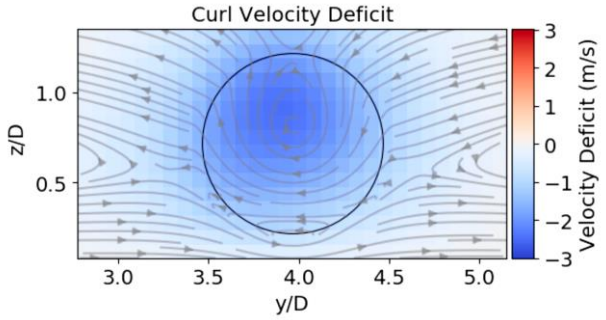
Ref: [4]

Large Array Updates

- Initial implementation of FLORIS as an expanded multizone version of the **Jensen wake deficit model**, coupled to the **Jimenez model** of wake deflection
- **Issue:** wake expansion was not affected by changes in control or turbulence
- **Gaussian wake deficit** and velocity models of as options within FLORIS provided **thrust and turbulence dependence**, and this Gaussian model became the **default wake model used within FLORIS**

Counter-rotating Vortices in Wake Steering

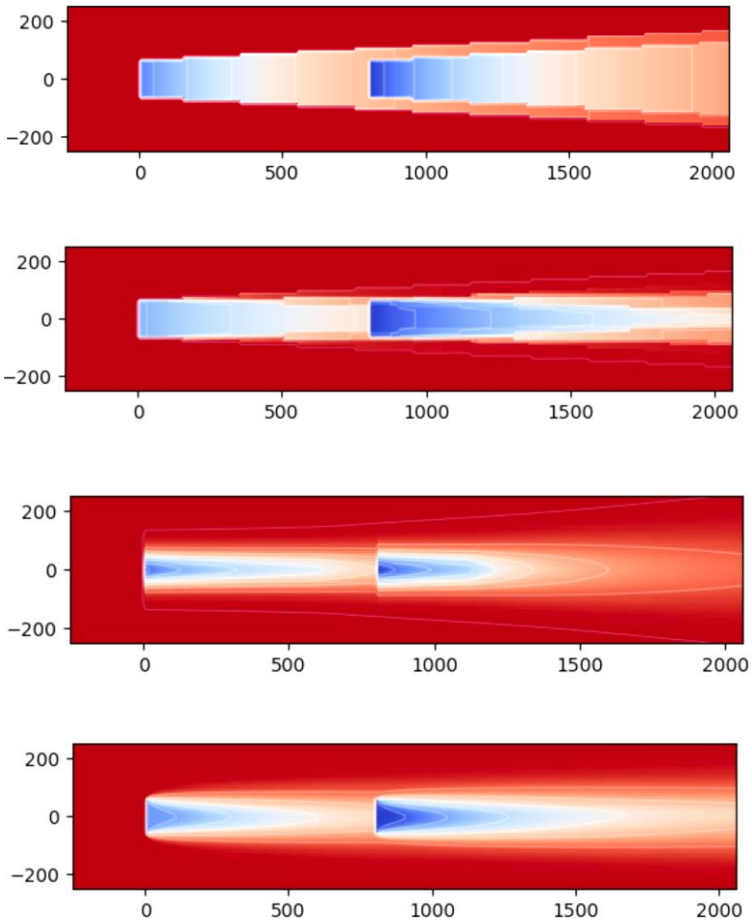
- An engineering model of **wake steering**, including these counter-rotating vortices, called **curl model**, is implemented into FLORIS in
- **Curled wake** model is **more computationally expensive** than **analytical models**, such as the **Gaussian** model
- Fast analytical models with low computational costs, which can be computed in fractions of a second, are **very important** for the suite of engineering processes in wind plant analysis, the design of wake control strategies, and **layout optimization**
- To implement the flow physics observed in secondary steering **without increasing the computational costs** of wake modeling, a new **Gauss-Curl Hybrid (GCH)** model was implemented
  - First modification, called **yaw-added recovery**
  - Second modification is **secondary steering**



Ref: [4]

Wake Models in FLORIS

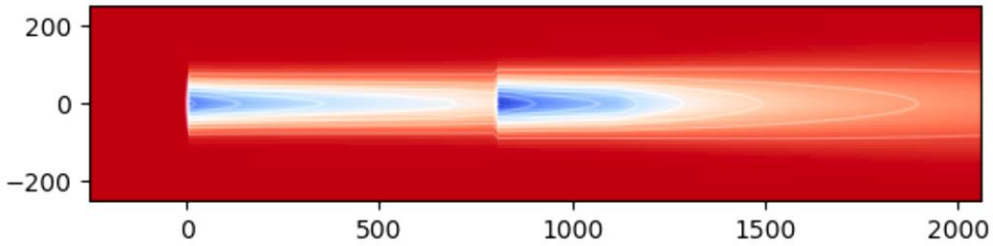
- Jensen (Park) Model – 0.0018 s
- Multi-zone wake model – 0.0019 s
- Gaussian wake model – 0.0025 s
- Curl model – 1.6 s

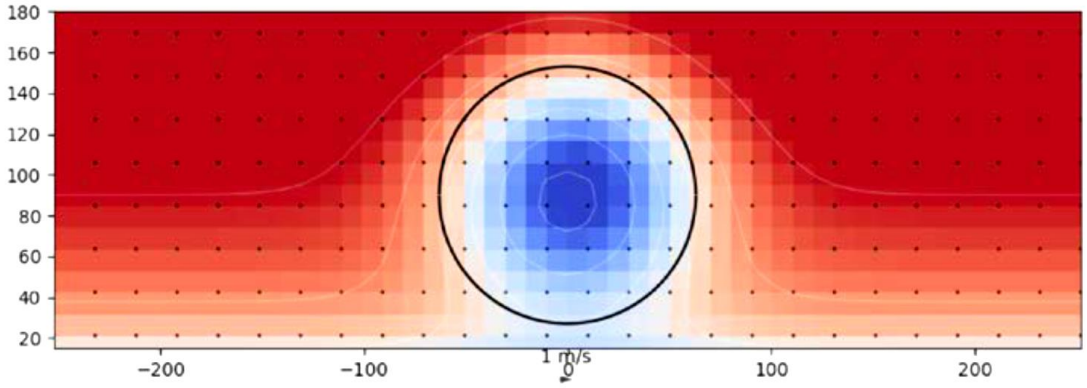


Ref: NREL

Wake Models – Gaussian

- Analytical solution to the simplified linearized Navier-Stokes equations
- Dependent on physical parameters that can be measured in the field
  - Ambient turbulence intensity
  - Shear
  - Veer
- Only **4** tuning parameters
- Good for normal turbine operation

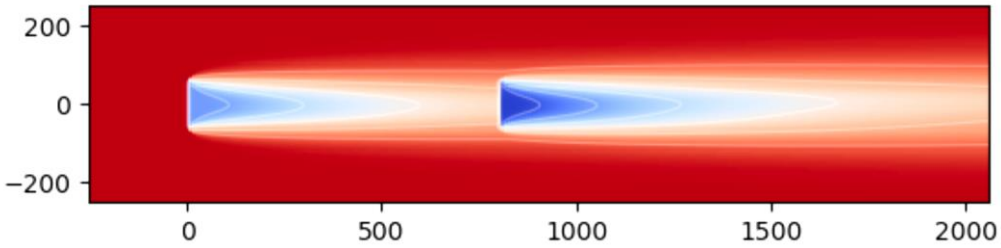


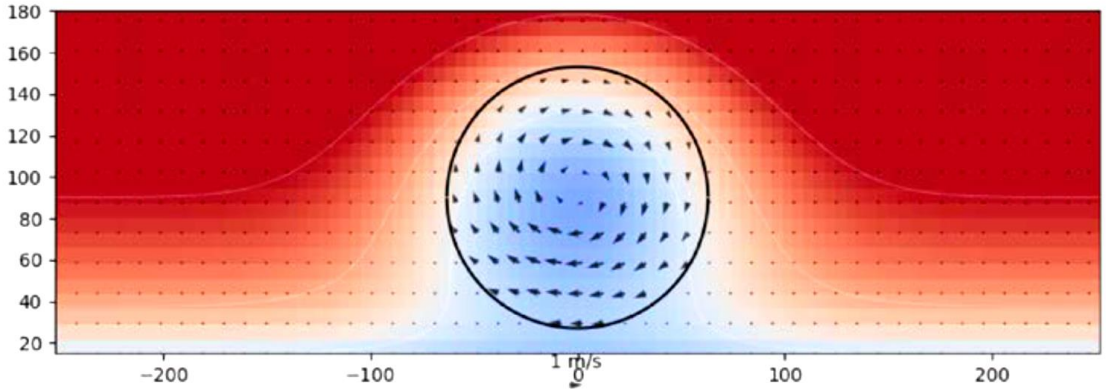


Ref: NREL

Wake Models – Curl

- Solves the linearized Navier-Stokes equations in time marching fashion
- Dependent on physical parameters that can be measured in the field
  - Ambient turbulence intensity
  - Shear
  - Veer
- Only **2** tuning parameters
- Good for **wake steering** analysis

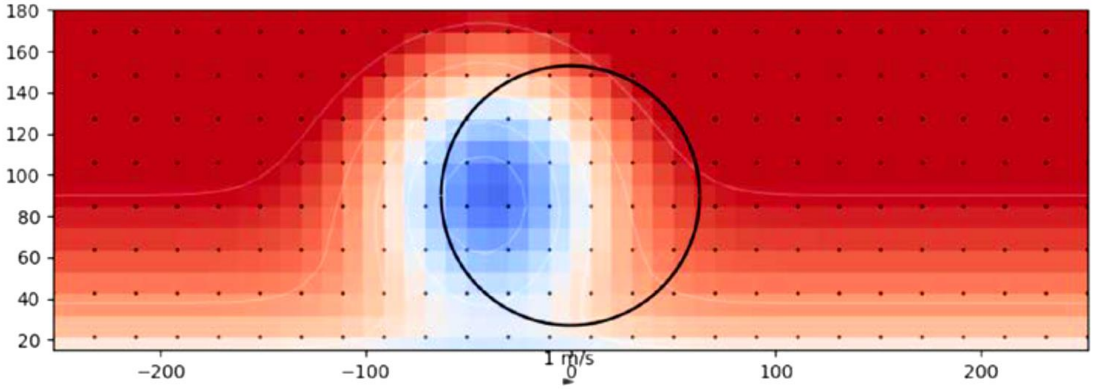




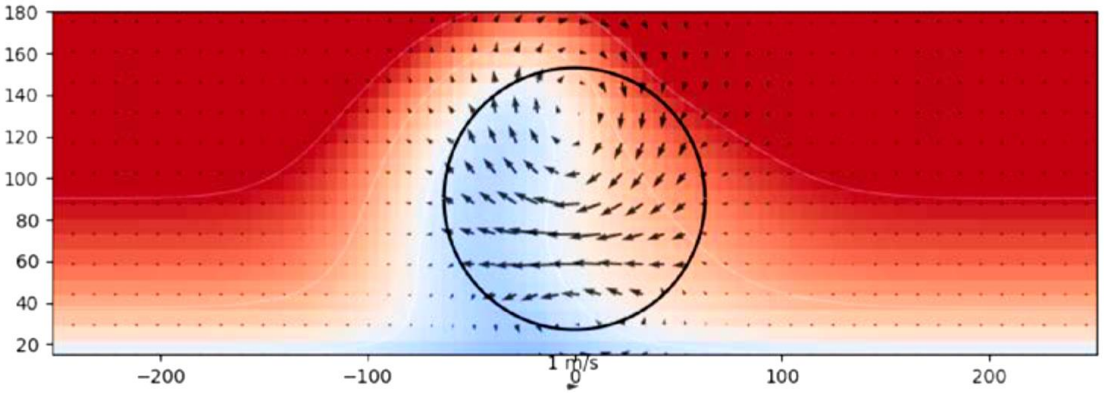
Ref: NREL

Deflection Model

- Gaussian model deflection
  - Wake is offset from centreline



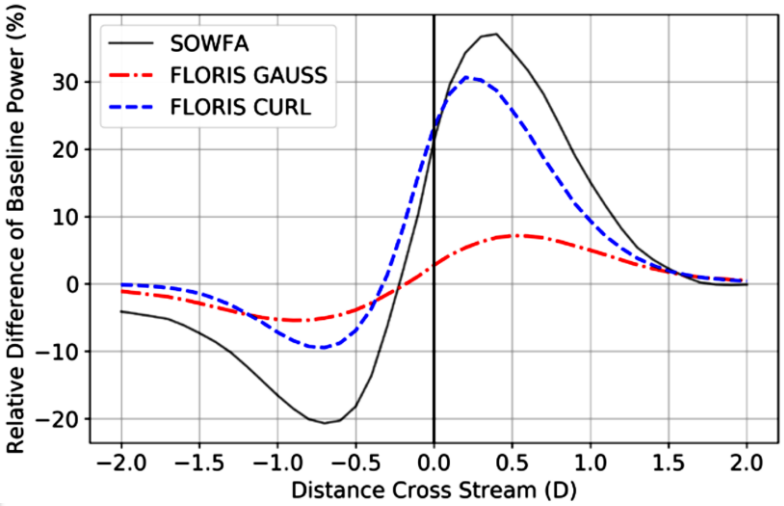
- Curl model deflection
  - Counter-rotating vortices
  - Wake rotation
  - Secondary steering

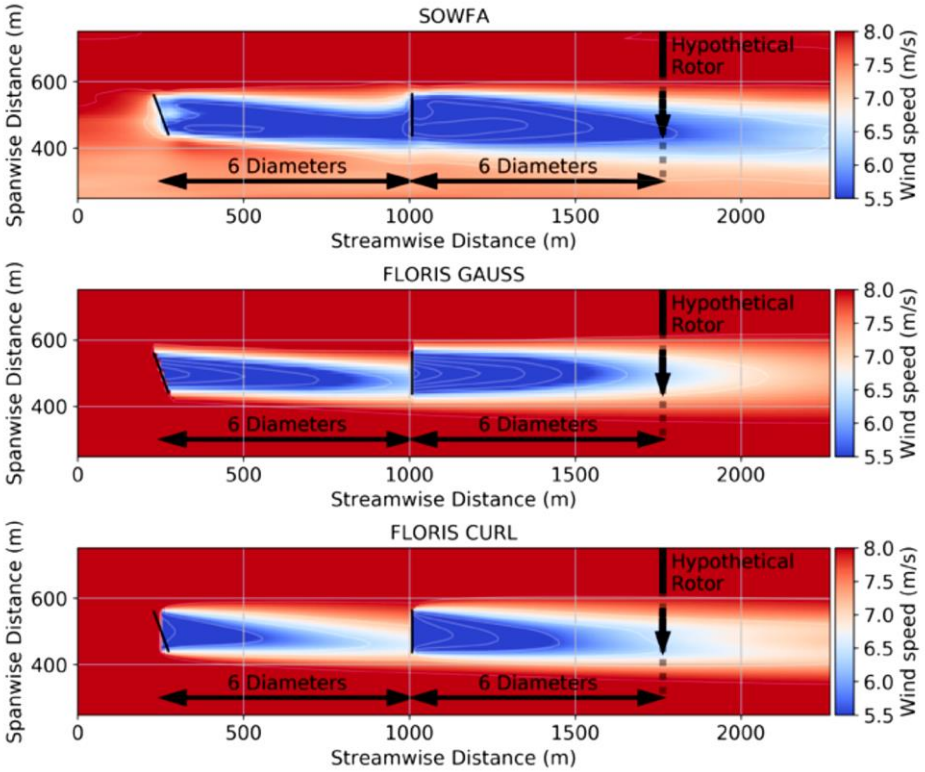


Ref: NREL

Compare Wake Steering

- Relative power difference 6D downstream of 2nd turbine

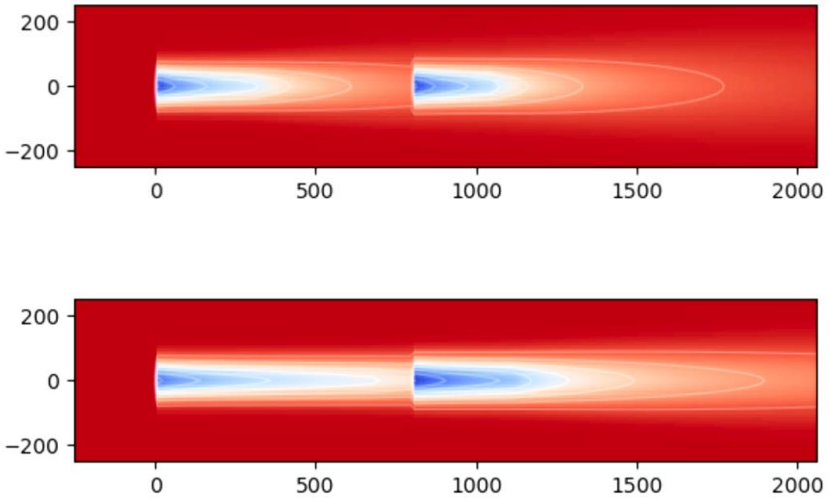




Ref: NREL

Turbulence Models
Turbine Model – Cp/Ct Tables

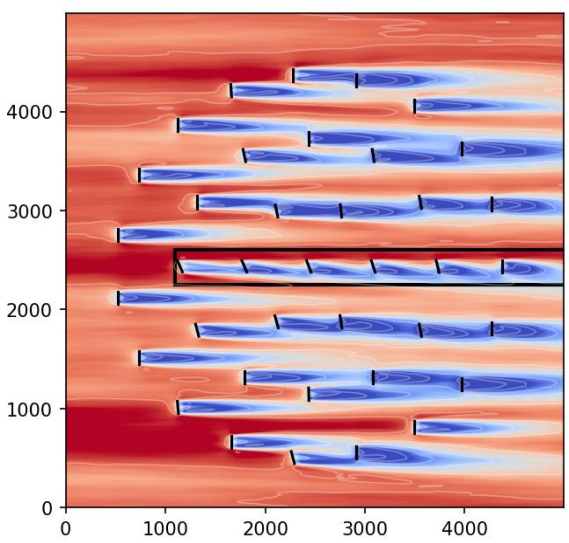
- Wake expansion dependent on ambient turbulence intensity
- Added turbulence due to turbine operation
  - As C_t increases, wake expansion increases
- Very important for investigating deep array effects (ongoing work)
- Turbine represented as **Actuator Disks**
- Generate C_p/C_t tables by:
  - *FAST* –Aeroelastic code
  - *CCBlade* – steady/state BEM coupled to FLORIS



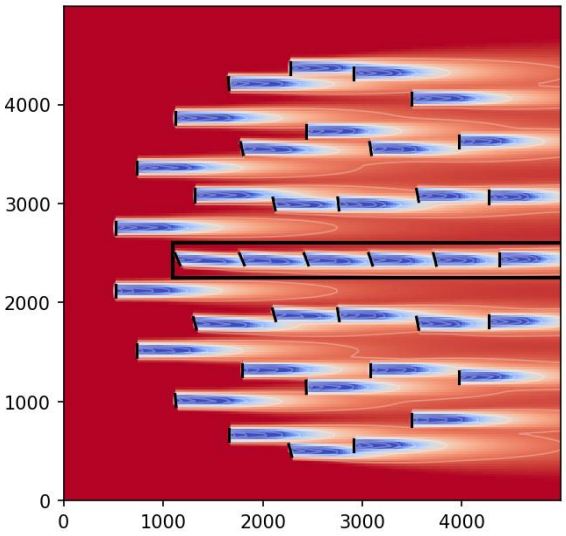
Ref: NREL

Heterogeneous Inflows

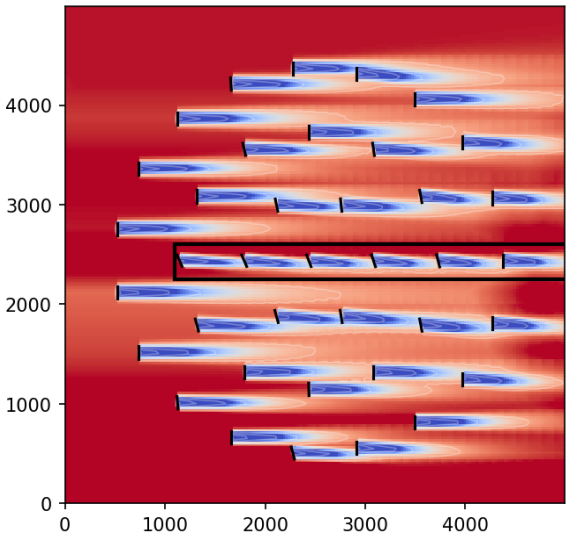
- Models the atmospheric in flow as a **heterogeneous field** with spatially varying wind speed, direction, and turbulence intensity
  - ✓ This improvement is now included by default, with homogeneous in flows being a special case of general heterogeneously defined in flows.



SOWFA - yawed



Homogeneous FLORIS - yawed



Heterogeneous FLORIS - yawed

- The gains in power from yawing the turbines is much better matched with the inclusion of secondary steering from GCH (Gauss-Curl Hybrid).

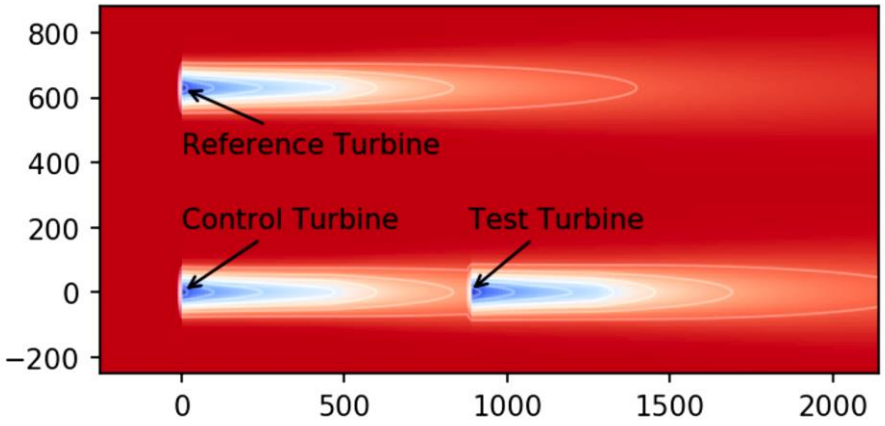
Ref: [4]

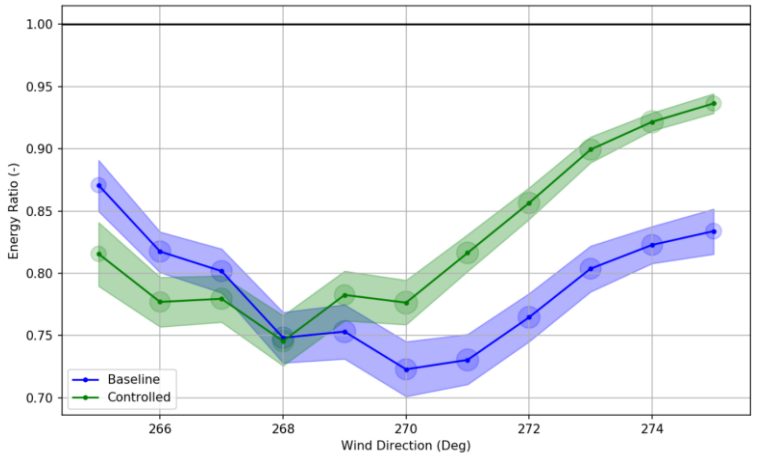
Optimization

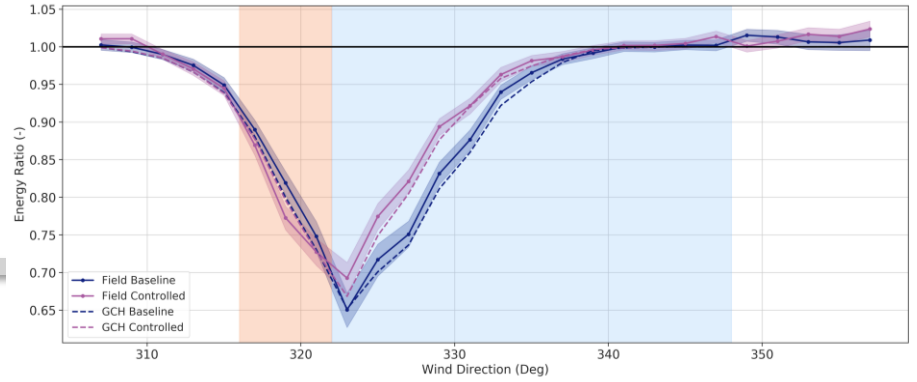
- The **FLORIS** framework includes methods for **design of wind farm control strategies** as well as **analysis**.
- Current update to FLORIS: robust methods for computing the **dynamic optimum**, by assuming **uncertainty in measurements**, as well as accounting for the **dynamics of the yaw controller**
- The effects of wind direction and yaw position uncertainty resulting from dynamic wind conditions are modeled in FLORIS by including a **distribution of wind directions and yaw off sets centered on the intended wind direction/yaw offset combination** when calculating wind farm power
- FLORIS users can provide their **own probability distributions for wind direction and yaw uncertainty** or rely on the **default Gaussian distributions**

Field Validation

- The **FLORIS** framework includes methods for **design of wind farm control strategies** as well as **analysis**.
- Often, FLORIS and field results are compared using a method called the **energy ratio.** The energy ratio function is included in FLORIS to facilitate open collaboration on analysis in a similar way as the open wake models.

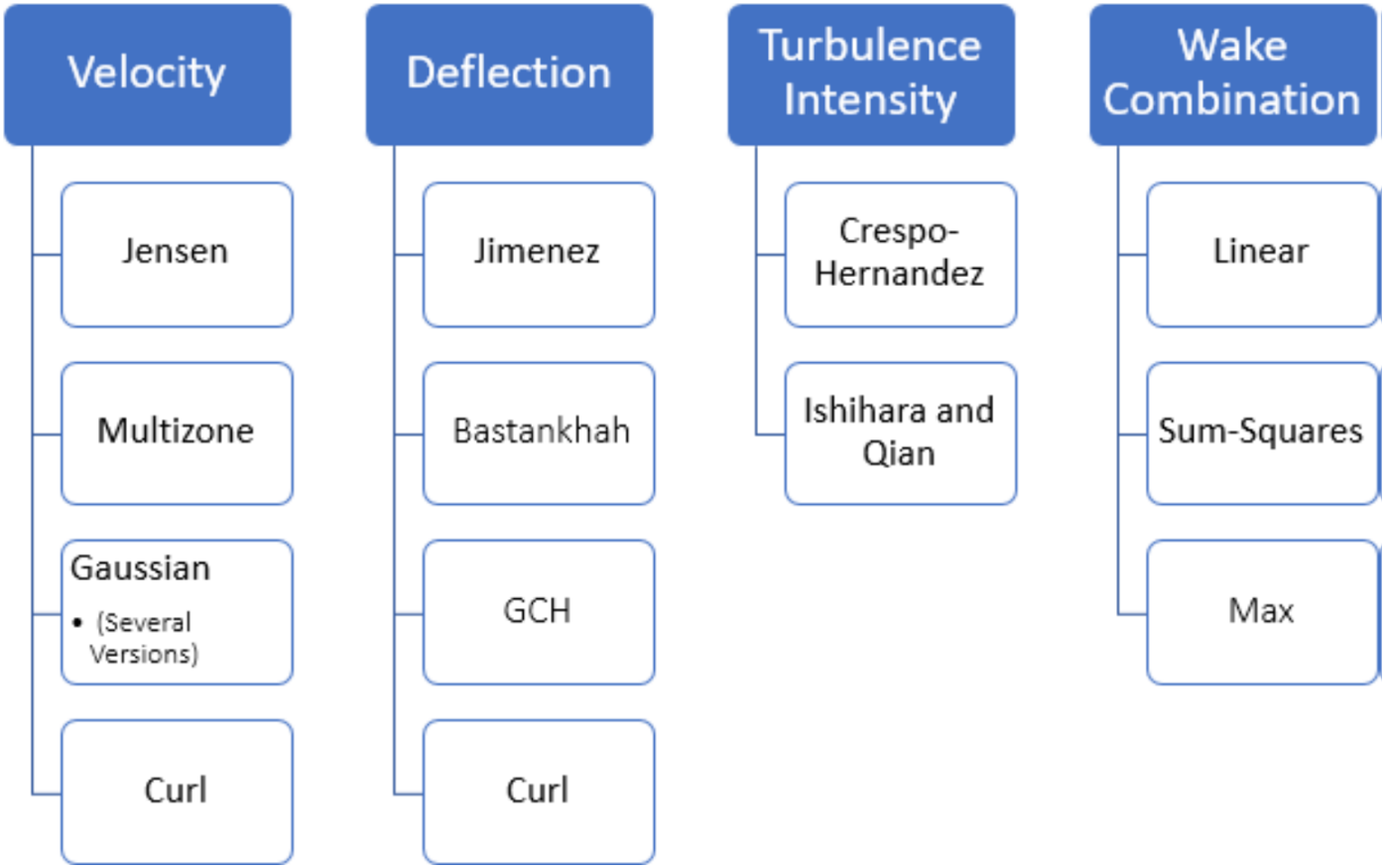






Ref: [4]

FLORIS Framework Modularization



Ref: [4]

FLORIS: Open-source and Collaborative

Available at:
https://github.com/NREL/floris

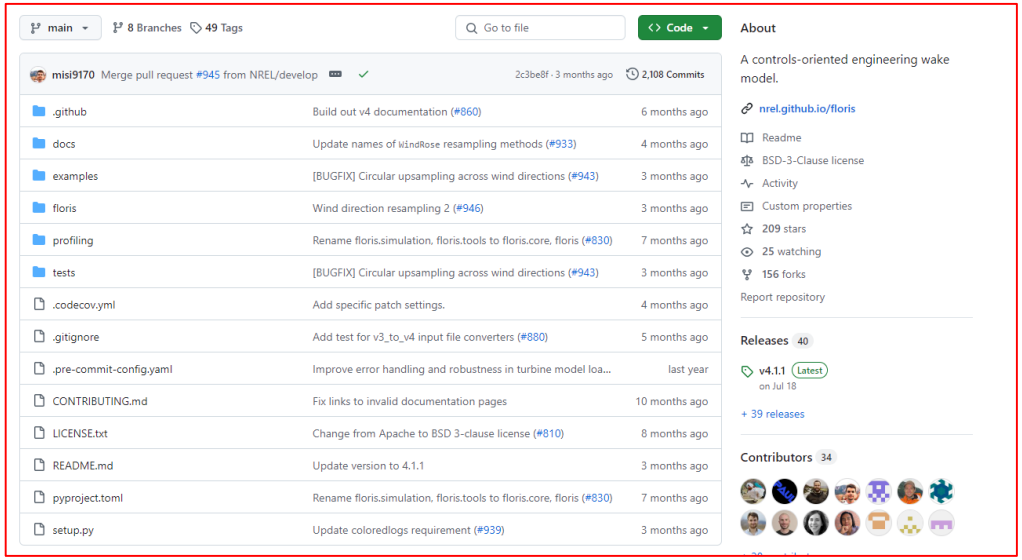
Divided into two packages:

- Simulation:
  - Contains code for FLORIS models
- Tools:
  - Modules for interacting with FLORIS models and data

Documentation and examples available at:
https://floris.readthedocs.io/_/downloads/en/latest/pdf/

http://floris.readthedocs.io/

https://nrel.github.io/floris

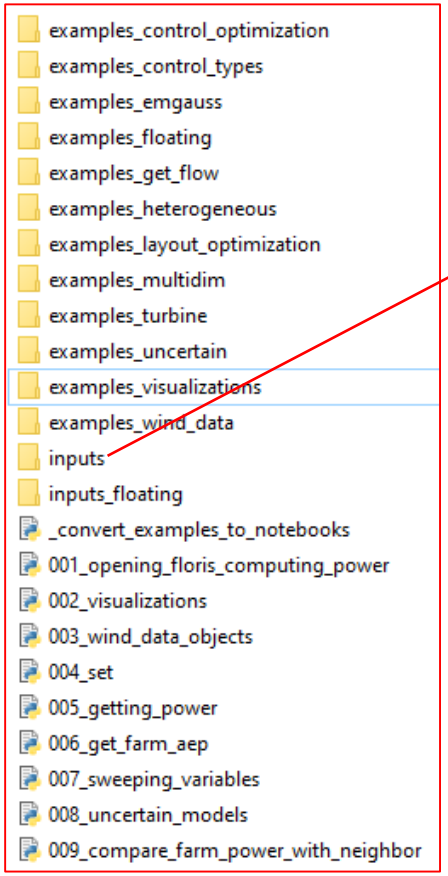


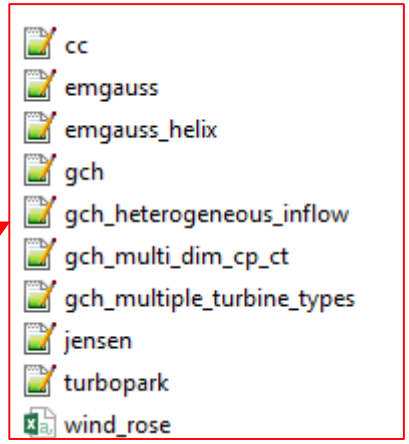
Ref: NREL

FLORIS Installation Folder

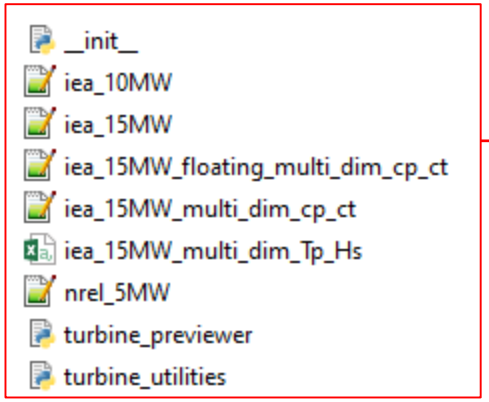
In the FLORIS installation folder, there are a couple of useful examples which are the main codes we use in this praktikum.

,,,\floris-main\examples





\floris-main\floris\turbine_library



You can add new turbine by having Power-Ct-Speed data

Example_1: Open and Calculate Power in FLORIS

Tools module allows for easy and intuitive interaction with FLORIS models.

All in python using open-source python modules.

001_opening_floris_computing_power.py

- **Line 17: import the FLORIS module**
- **Line 22: create FLORIS interface**
- **Line 25: define farm layout**
- **Line 30: define wind directions and wind speeds**
- **Line 55: calculate turbine power**
- **Line 56: calculate farm power**

```
15  import numpy as np
16
17  from floris import FlorisModel
18
19
20  # The FlorisModel class is the entry point for most usage.
21  # Initialize using an input yaml file
22  fmodel = FlorisModel("inputs/gch.yaml")
23
24  # Changing the wind farm layout uses FLORIS' set method to a two-turbine layout
25  fmodel.set(layout_x=[0, 500.0], layout_y=[0.0, 0.0])
26
27  # Changing wind speed, wind direction, and turbulence intensity uses the set method
28  # as well. Note that the wind_speeds, wind_directions, and turbulence_intensities
29  # are all specified as arrays of the same length.
30  fmodel.set(
31      wind_directions=np.array([270.0]), wind_speeds=[8.0], turbulence_intensities=np.array([0.06])
32  )
33
34  # Note that typically all 3, wind_directions, wind_speeds and turbulence_intensities
35  # must be supplied to set.  However, the exception is if not changing the length
36  # of the arrays, then only one or two may be supplied.
37  fmodel.set(turbulence_intensities=np.array([0.07]))
38
39  # The number of elements in the wind_speeds, wind_directions, and turbulence_intensities
40  # corresponds to the number of conditions to be simulated.  In FLORIS, each of these are
41  # tracked by a simple index called a findex.  There is no requirement that the values
42  # be unique.  Internally in FLORIS, most data structures will have the findex as their
43  # 0th dimension.  The value n_findex is the total number of conditions to be simulated.
44  # This command would simulate 4 conditions (n_findex = 4).
45  fmodel.set(
46      wind_directions=np.array([270.0, 270.0, 270.0, 270.0]),
47      wind_speeds=[8.0, 8.0, 10.0, 10.0],
48      turbulence_intensities=np.array([0.06, 0.06, 0.06, 0.06]),
49  )
50
51  # After the set method, the run method is called to perform the simulation
52  fmodel.run()
53
54  # There are functions to get either the power of each turbine, or the farm power
55  turbine_powers = fmodel.get_turbine_powers() / 1000.0
56  farm_power = fmodel.get_farm_power() / 1000.0
57
58  print("The turbine power matrix should be of dimensions 4 (n_findex) X 2 (n_turbines)")
59  print(turbine_powers)
60  print("Shape: ", turbine_powers.shape)
```

FLORIS Interface

Many important input files should be defined in the interface file (.yaml)

```yaml
farm:

  ###
  # Coordinates for the turbine locations in the x-direction which is typically considered
  # to be the streamwise direction (left, right) when the wind is out of the west.
  # The order of the coordinates here corresponds to the index of the turbine in the primary
  # data structures.
  layout_x:
  - 0.0
  - 630.0
  - 1260.0

  ###
  # Coordinates for the turbine locations in the y-direction which is typically considered
  # to be the spanwise direction (up, down) when the wind is out of the west.
  # The order of the coordinates here corresponds to the index of the turbine in the primary
  # data structures.
  layout_y:
  - 0.0
  - 0.0
  - 0.0

  ###
  # Listing of turbine types for placement at the x and y coordinates given above.
  # The list length must be 1 or the same as ``layout_x`` and ``layout_y``. If it is a
  # single value, all turbines are of the same type. Otherwise, the turbine type
  # is mapped to the location at the same index in ``layout_x`` and ``layout_y``.
  # The types can be either a name included in the turbine_library or
  # a full definition of a wind turbine directly.
  turbine_type:
  - nrel_5MW

###
# Configure the atmospheric conditions.
flow_field:

  ###
  # Air density.
  air_density: 1.225
```

- **It should be noted that some of these parameters can be also defined in the code**

Example_2: Calculate Wind Farm AEP

In the example the Annual Energy Production (AEP) of a wind farm using wind rose information stored in a .csv file can be calculated.

006_get_farm_aep.py

- Line 25: create FLORIS interface
- Line 29-31: define farm layout
- Line 41: import time series
- Line 62: import the wind rose
- Line 88: calculate AEP

- Wind rose defined by three parameters:
  - ws (wind speed)
  - wd (wind direction)
  - freq_val (frequency)

```
Total number of wind direction and wind speed combination: 1872
Number of 0 frequency bins: 545
n_findex: 1327
AEP from wind rose: 59.076 (GWh)
Wake losses: 0.99%
```

	A	B	C
1	ws	wd	freq_val
2	0	0	1.63E-05
3	0	5	1.63E-05
4	0	10	4.89E-05
5	0	15	3.26E-05
6	0	20	6.52E-05
7	0	25	6.52E-05
8	0	30	1.63E-05
9	0	35	0
10	0	40	3.26E-05
11	0	45	4.89E-05
12	0	50	0
13	0	55	0
14	0	60	4.89E-05
15	0	65	1.63E-05
16	0	70	3.26E-05
17	0	75	1.63E-05
18	0	80	3.26E-05
19	0	85	4.89E-05
20	0	90	4.89E-05
21	0	95	4.89E-05
22	0	100	4.89E-05
23	0	105	6.52E-05
24	0	110	4.89E-05
25	0	115	4.89E-05

```python
import numpy as np
import pandas as pd

from floris import (
    FlorisModel,
    TimeSeries,
    WindRose,
)


fmodel = FlorisModel("inputs/gch.yaml")


# Set to a 3-turbine layout
D = 126.
fmodel.set(layout_x=[0.0, 5 * D, 10 * D],
           layout_y=[0.0, 0.0, 0.0])

# Using TimeSeries

# Randomly generated a time series with time steps = 365 * 24
N = 365 * 24
wind_directions = np.random.uniform(0, 360, N)
wind_speeds = np.random.uniform(5, 25, N)

# Set up a time series
time_series = TimeSeries(
    wind_directions=wind_directions, wind_speeds=wind_speeds, turbulence_intensities=0.06
)

# Set the wind data
fmodel.set(wind_data=time_series)

# Run the model
fmodel.run()
```

```python
# Using WindRose==============================================

# Load the wind rose from csv as in example 003
wind_rose = WindRose.read_csv_long(
    "inputs/wind_rose.csv", wd_col="wd", ws_col="ws", freq_col="freq_val", ti_col_or_value=0.06
)


# Store some values
n_wd = len(wind_rose.wind_directions)
n_ws = len(wind_rose.wind_speeds)

# Store the number of elements of the freq_table which are 0
n_zeros = np.sum(wind_rose.freq_table == 0)

# Set the wind rose
fmodel.set(wind_data=wind_rose)

# Run the model
fmodel.run()

# Note that the frequency table contains 0 frequency for some wind directions and wind speeds
# and we've not selected to compute 0 frequency bins, therefore the n_findex will be less than
# the total number of wind directions and wind speed combinations
print(f"Total number of wind direction and wind speed combination: {n_wd * n_ws}")
print(f"Number of 0 frequency bins: {n_zeros}")
print(f"n_findex: {fmodel.n_findex}")

# Get the AEP
aep = fmodel.get_farm_AEP()
```

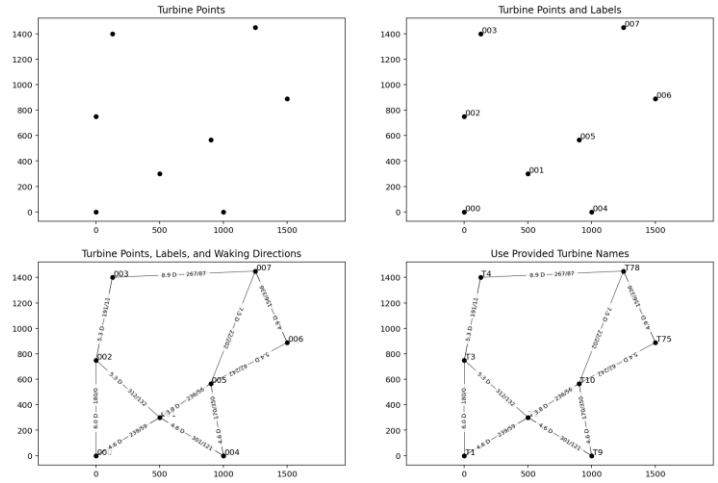
Praktikum Design of Wind Farms

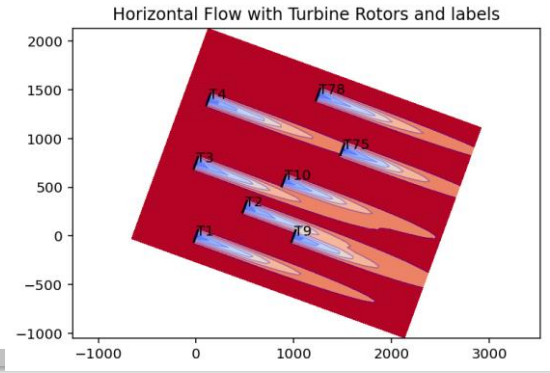
Wind Energy Institute

Example_3: Visualizations

This example initializes the FLORIS software, and then uses internal functions to run a simulation and plot the results.

002_visualizations.py






```python
import matplotlib.pyplot as plt

import floris.layout_visualization as layoutviz
from floris import FlorisModel
from floris.flow_visualization import visualize_cut_plane


fmodel = FlorisModel("inputs/gch.yaml")

# Set the farm layout to have 8 turbines irregularly placed
layout_x = [0, 500, 0, 128, 1000, 900, 1500, 1250]
layout_y = [0, 300, 750, 1400, 0, 567, 888, 1450]
fmodel.set(layout_x=layout_x, layout_y=layout_y)


# Layout visualization contains the functions for visualizing the layout:
# plot_turbine_points
# plot_turbine_labels
# plot_turbine_rotors
# plot_waking_directions
# Each of which can be overlaid to provide further information about the layout
# This series of 4 subplots shows the different ways to visualize the layout

# Create the plotting objects using matplotlib
fig, axarr = plt.subplots(2, 2, figsize=(15, 10), sharex=False)
axarr = axarr.flatten()

ax = axarr[0]
layoutviz.plot_turbine_points(fmodel, ax=ax)
ax.set_title("Turbine Points")

ax = axarr[1]
layoutviz.plot_turbine_points(fmodel, ax=ax)
layoutviz.plot_turbine_labels(fmodel, ax=ax)
ax.set_title("Turbine Points and Labels")
```

```python
fmodel.set(wind_speeds=[8.0], wind_directions=[290.0], turbulence_intensities=[0.06])
horizontal_plane = fmodel.calculate_horizontal_plane(
    x_resolution=200,
    y_resolution=100,
    height=90.0,
)

# Plot the flow field with rotors
fig, ax = plt.subplots()
visualize_cut_plane(
    horizontal_plane,
    ax=ax,
    label_contours=False,
    title="Horizontal Flow with Turbine Rotors and labels",
)

# Plot the turbine rotors
layoutviz.plot_turbine_rotors(fmodel, ax=ax)
layoutviz.plot_turbine_labels(fmodel, ax=ax, turbine_names=turbine_names)

plt.show()
```

Praktikum Design of Wind Farms

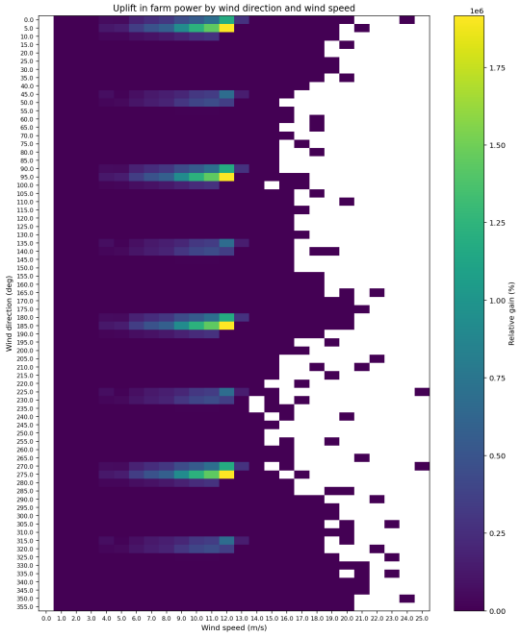
Wind Energy Institute

Example_4: Optimization

Perform yaw optimizations to investigate **wake steering** power gains.

004_optimize_yaw_aep.py

- Line 54-60: create optimization object with min. and max. yaw angles
- Line 62: perform yaw optimization




```
34  # Specify wind farm layout and update in the floris object
35  N = 2  # number of turbines per row and per column
36  X, Y = np.meshgrid(
37      5.0 * fmodel.core.farm.rotor_diameters_sorted[0][0] * np.arange(0, N, 1),
38      5.0 * fmodel.core.farm.rotor_diameters_sorted[0][0] * np.arange(0, N, 1),
39  )
40  fmodel.set(layout_x=X.flatten(), layout_y=Y.flatten())
41
42  # Get the number of turbines
43  n_turbines = len(fmodel.layout_x)
44
45  # Optimize the yaw angles.  This could be done for every wind direction and wind speed
46  # but in practice it is much faster to optimize only for one speed and infer the rest
47  # using a rule of thumb
48  time_series = TimeSeries(
49      wind_directions=wind_rose.wind_directions, wind_speeds=8.0, turbulence_intensities=0.06
50  )
51  fmodel.set(wind_data=time_series)
52
53  # Get the optimal angles
54  start_time = timerpc()
55  yaw_opt = YawOptimizationSR(
56      fmodel=fmodel,
57      minimum_yaw_angle=0.0,  # Allowable yaw angles lower bound
58      maximum_yaw_angle=20.0,  # Allowable yaw angles upper bound
59      Ny_passes=[5, 4],
60      exclude_downstream_turbines=True,
61  )
62  df_opt = yaw_opt.optimize()
63  end_time = timerpc()
64  t_tot = end_time - start_time
65  print("Optimization finished in {:.2f} seconds.".format(t_tot))
66
```

```
Baseline AEP: 75.48 GWh.
Optimal AEP: 76.18 GWh.
Relative AEP uplift by wake steering: 0.926 %.
```

Praktikum Design of Wind Farms

Wind Energy Institute

Ref: NREL

References

1) Nicholas Hamilton, Christopher J. Bay, Paul Fleming, et al. Comparison of modular analytical wake models to the Lillgrund wind plant, J. Renewable Sustainable Energy 12, 053311 (2020)

2) Alfredo Peña, Pierre-Elouan Réthoré and M. Paul van der Laan, On the application of the Jensen wake model using a turbulence-dependent wake decay coefficient: the Sexbierum case, Wind Energ. 2016; 19:763–776.

3) Majid Bastankhah, Fernando Porté-Agel, A new analytical model for wind-turbine wakes, Renewable Energy 70 (2014) 116-123.

4) Paul Fleming, Jennifer King, Christopher J. Bay, Eric Simley, Rafael Mudafort, Nicholas Hamilton, Alayna Farrell, and Luis Martinez-Tossas, Overview of FLORIS updates, Journal of Physics: Conference Series 1618 (2020), 022028.